

Final Exam Review

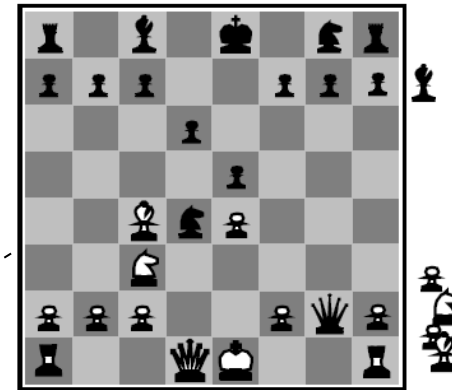
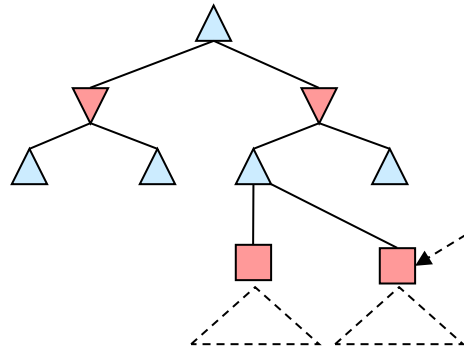
- Friday 10:30-12:30pm in class!
- Bring a calculator. You can check out one from the library.
- One page of notes front and back.

Look where we've been!

Informed Search:
A-Star

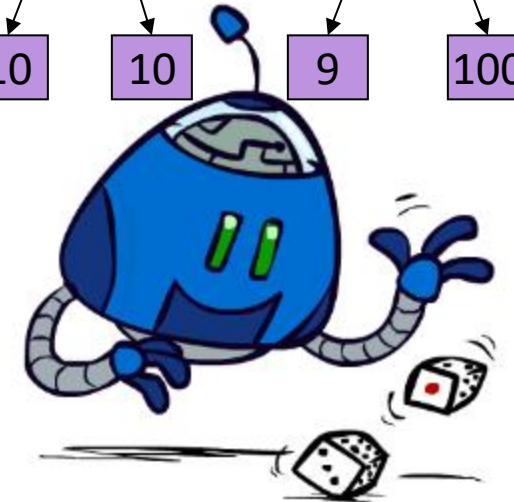
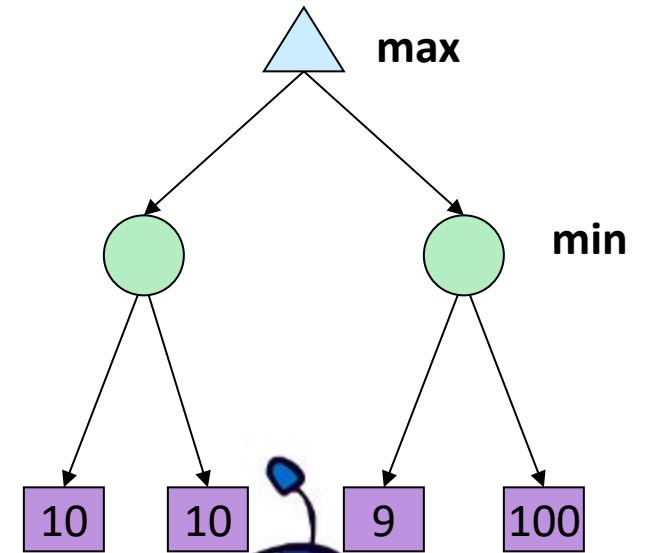


Adversarial Search:
Alpha-Beta Pruning



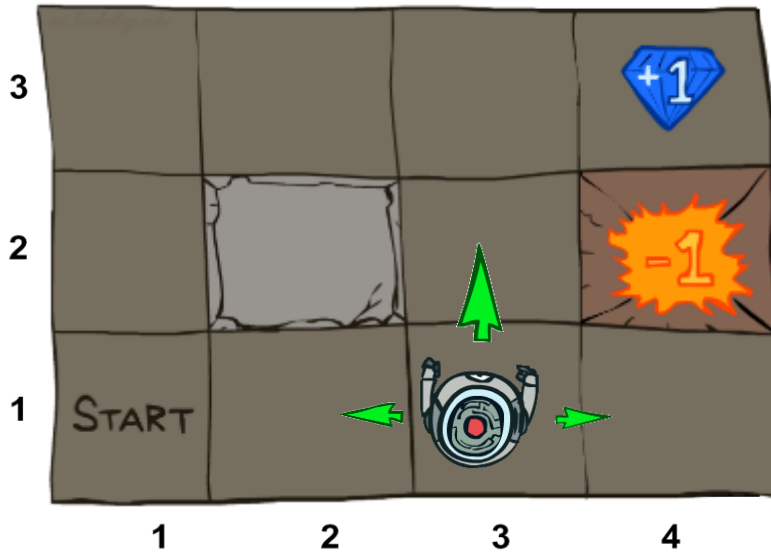
White to move
Black winning

Expectimax Search

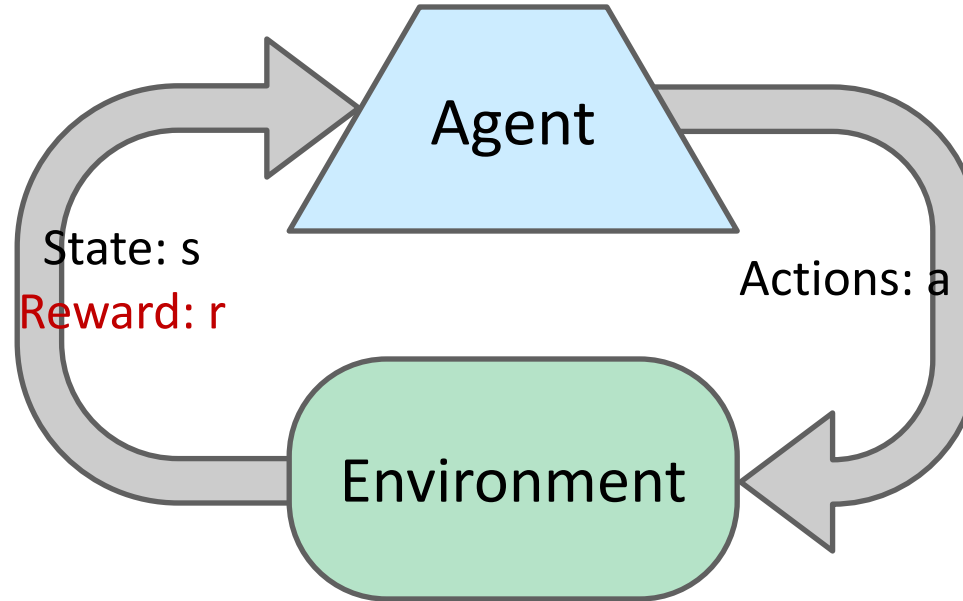


Look where we've been!

MDPs:
Value Iteration, Policy
Iteration



Reinforcement
Learning: Q-Learning,
Policy Gradients

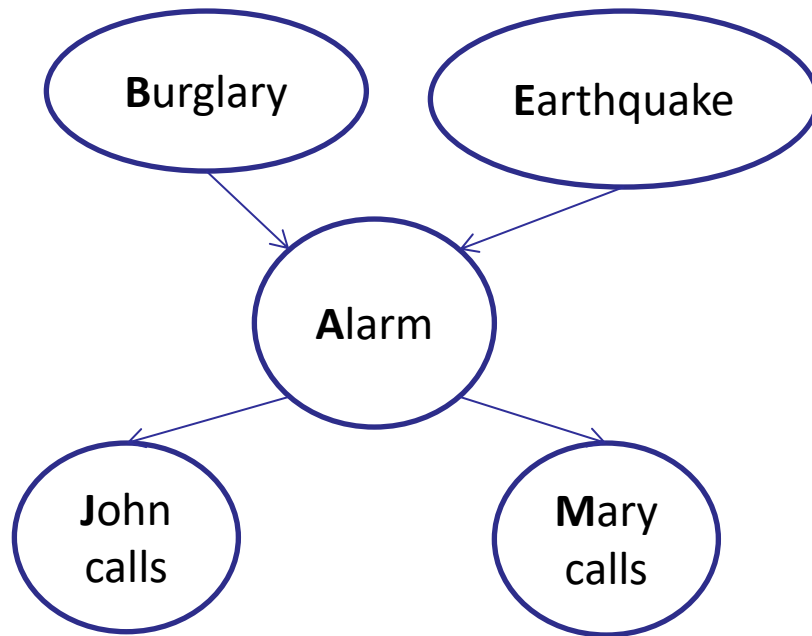


DQN
AlphaGo

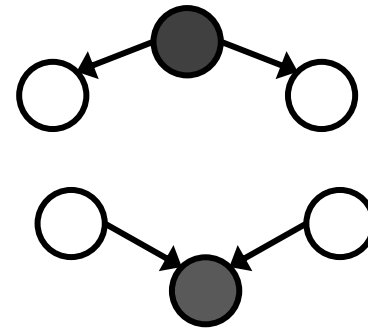


Look where we've been!

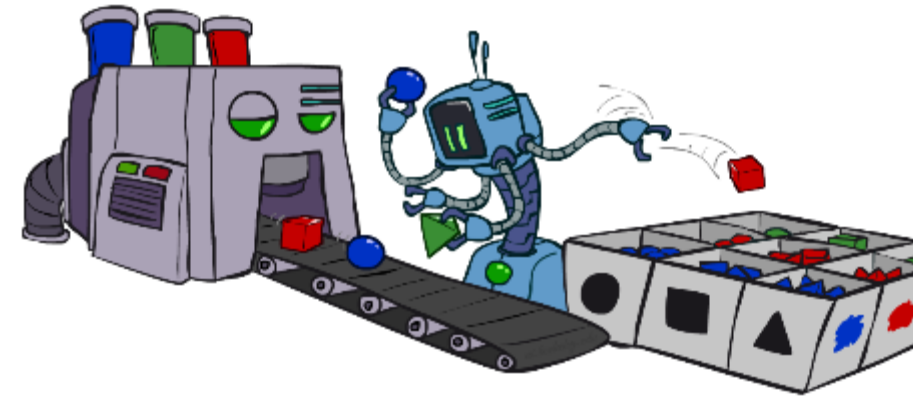
Bayes' Nets



D-Separation

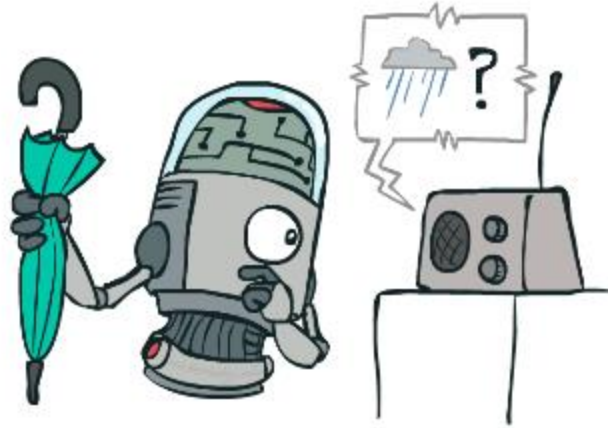
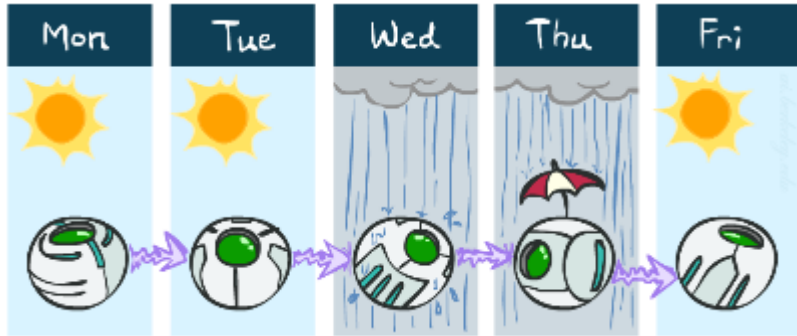


Sampling

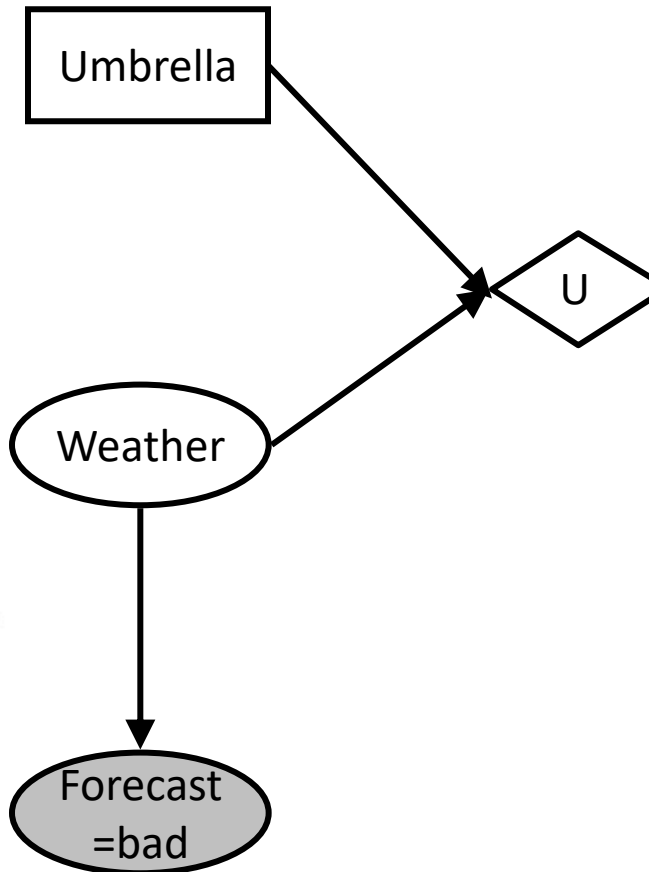


Look where we've been!

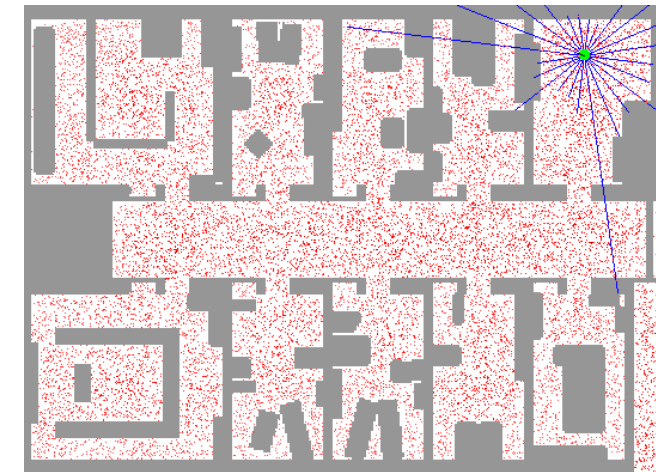
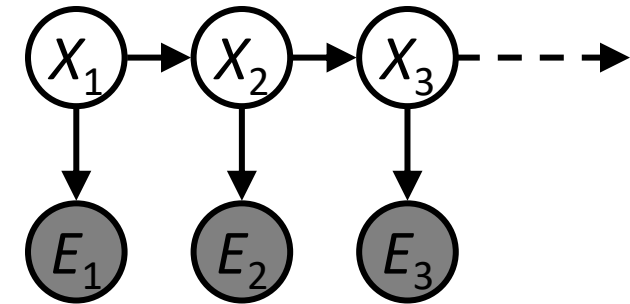
Markov Models



Value of Perfect Information

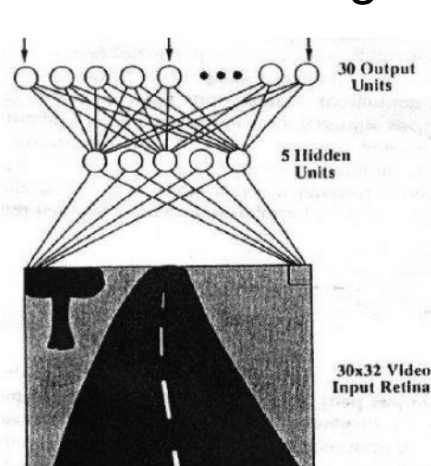


Hidden Markov Models: Particle Filters

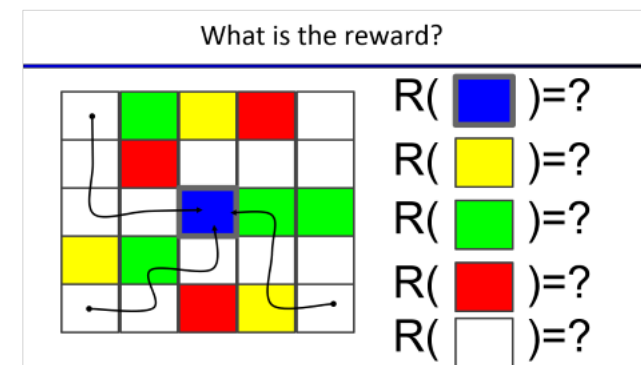


Look where we've been!

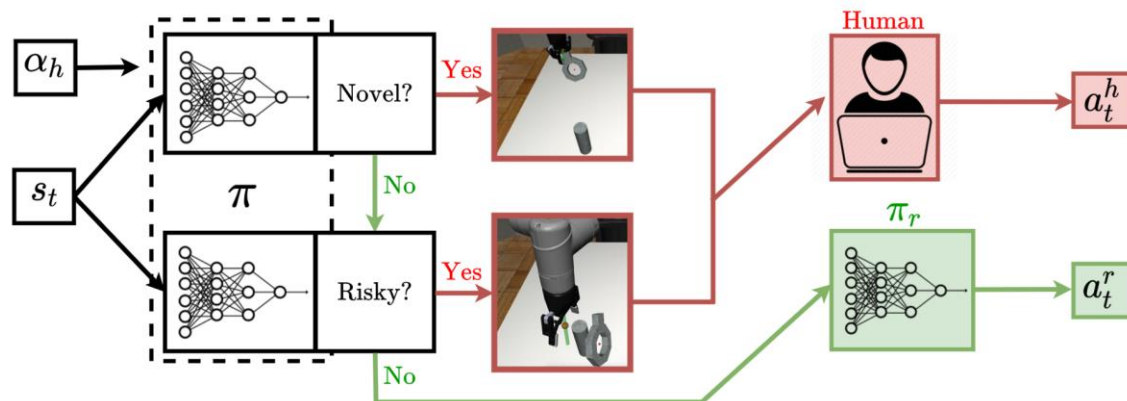
Behavioral Cloning



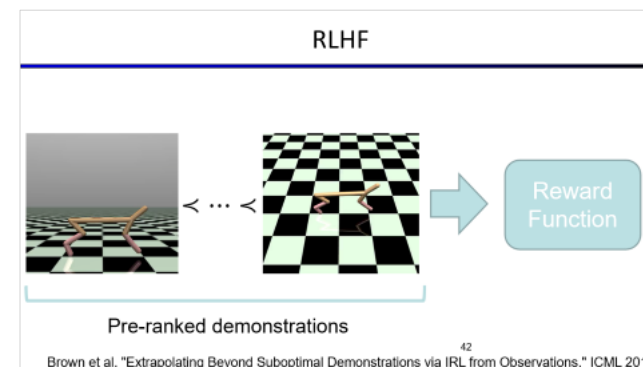
Inverse RL



DAgger



RL from Human Feedback



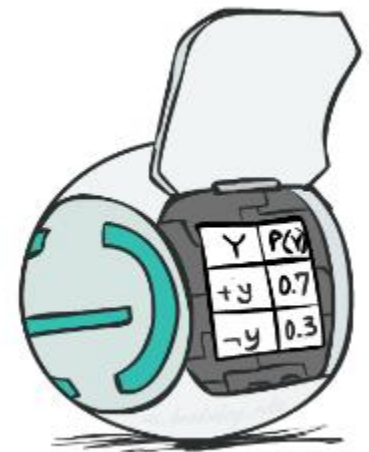
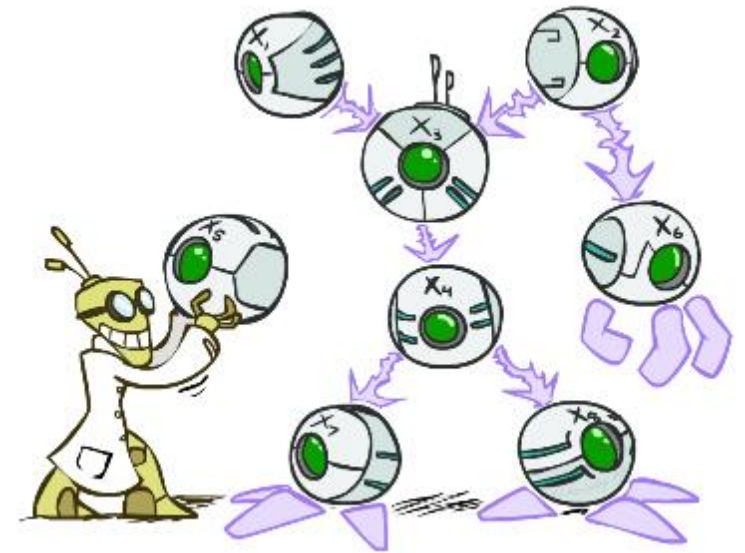
Bayes' Net Semantics

- A directed, acyclic graph, one node per random variable
- A conditional probability table (CPT) for each node
 - A collection of distributions over X , one for each combination of parents' values

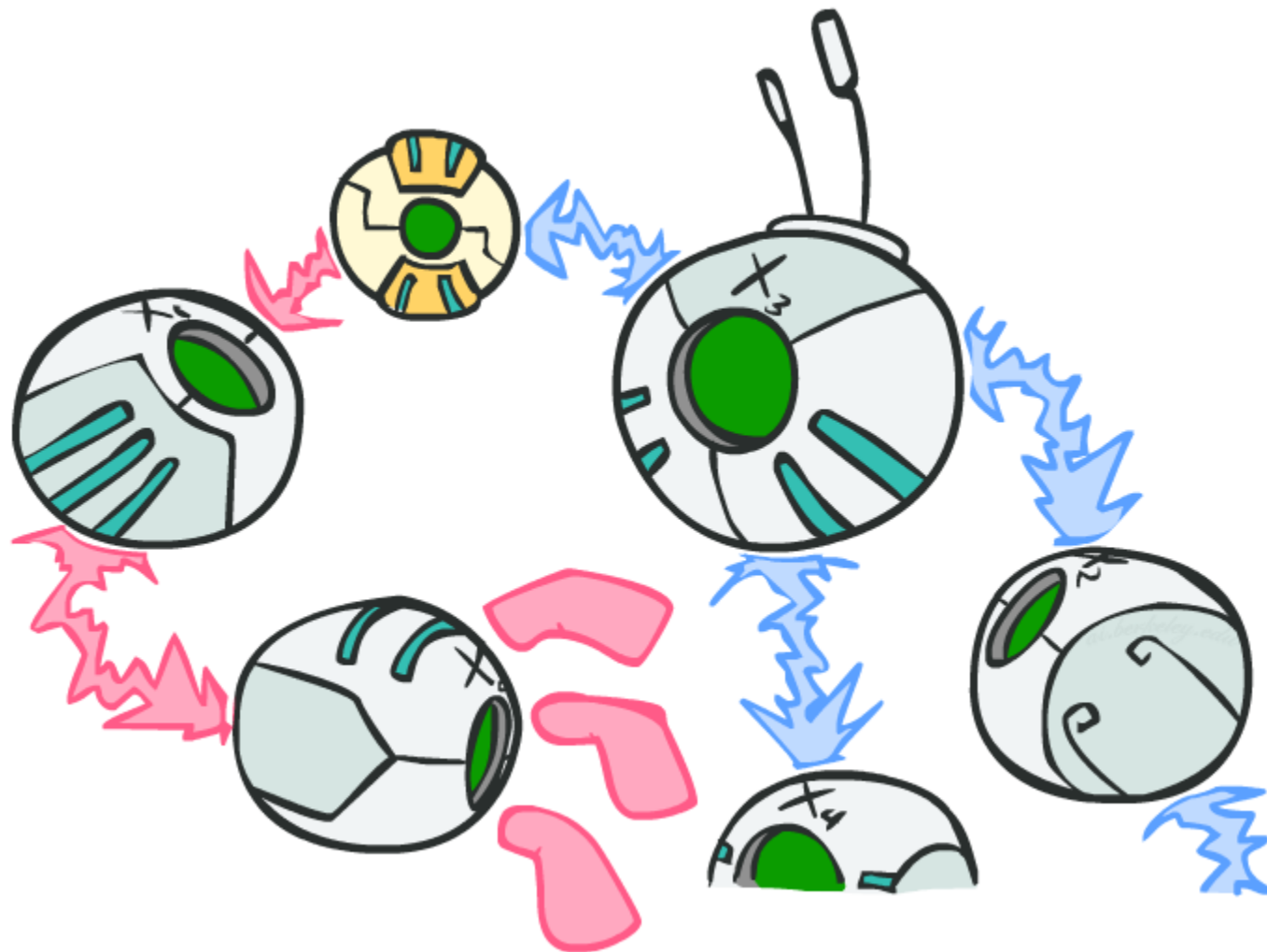
$$P(X|a_1 \dots a_n)$$

- Bayes' nets implicitly encode joint distributions
 - As a product of local conditional distributions
 - To see what probability a BN gives to a full assignment, multiply all the relevant conditionals together:

$$P(x_1, x_2, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(X_i))$$



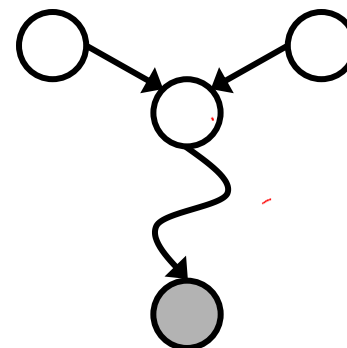
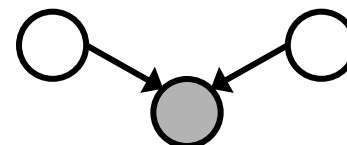
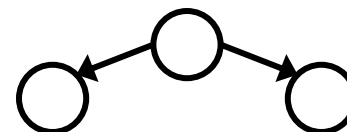
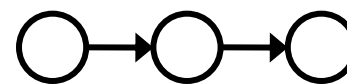
D-separation: Outline



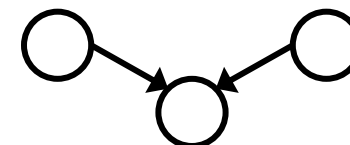
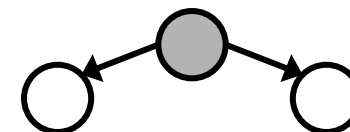
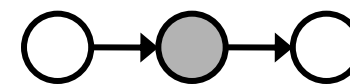
Active / Inactive Paths

- Question: Are X and Y conditionally independent given evidence variables {Z}?
 - Yes, if X and Y “d-separated” by Z
 - Consider all (undirected) paths from X to Y
 - No active paths = independence!
- A path is active if each triple is active:
 - Causal chain $A \rightarrow B \rightarrow C$ where B is unobserved (either direction)
 - Common cause $A \leftarrow B \rightarrow C$ where B is unobserved
 - Common effect (aka v-structure)
 $A \rightarrow B \leftarrow C$ where B or one of its descendants is observed
- All it takes to block a path is a single inactive segment

Active Triples



Inactive Triples



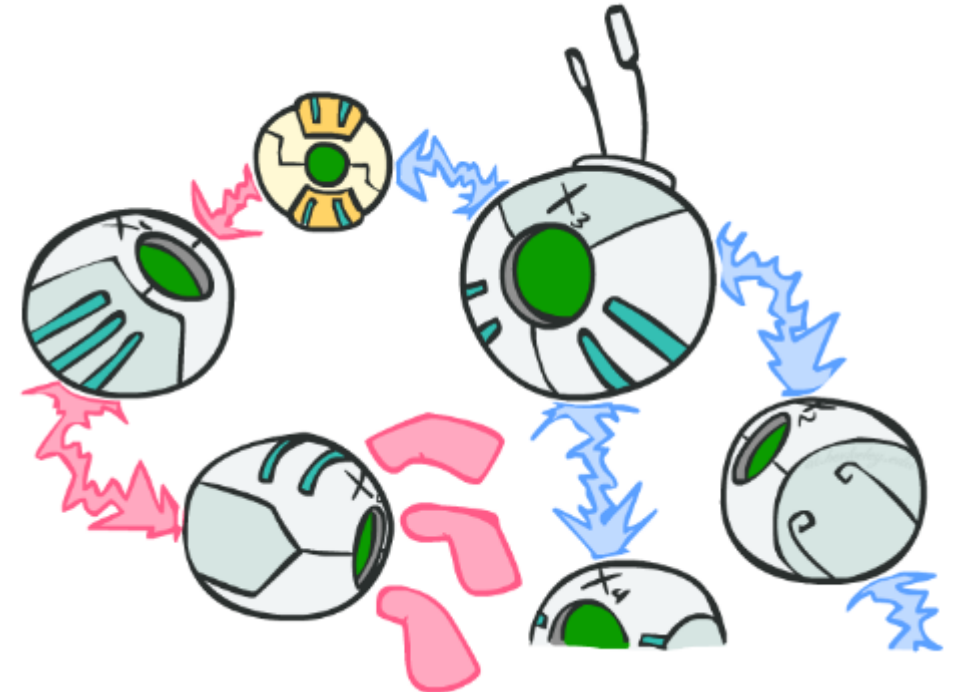
D-Separation

- Query: $X_i \perp\!\!\!\perp X_j \mid \{X_{k_1}, \dots, X_{k_n}\} ?$
- Check all (undirected!) paths between X_i and X_j
 - If one or more active, then independence not guaranteed

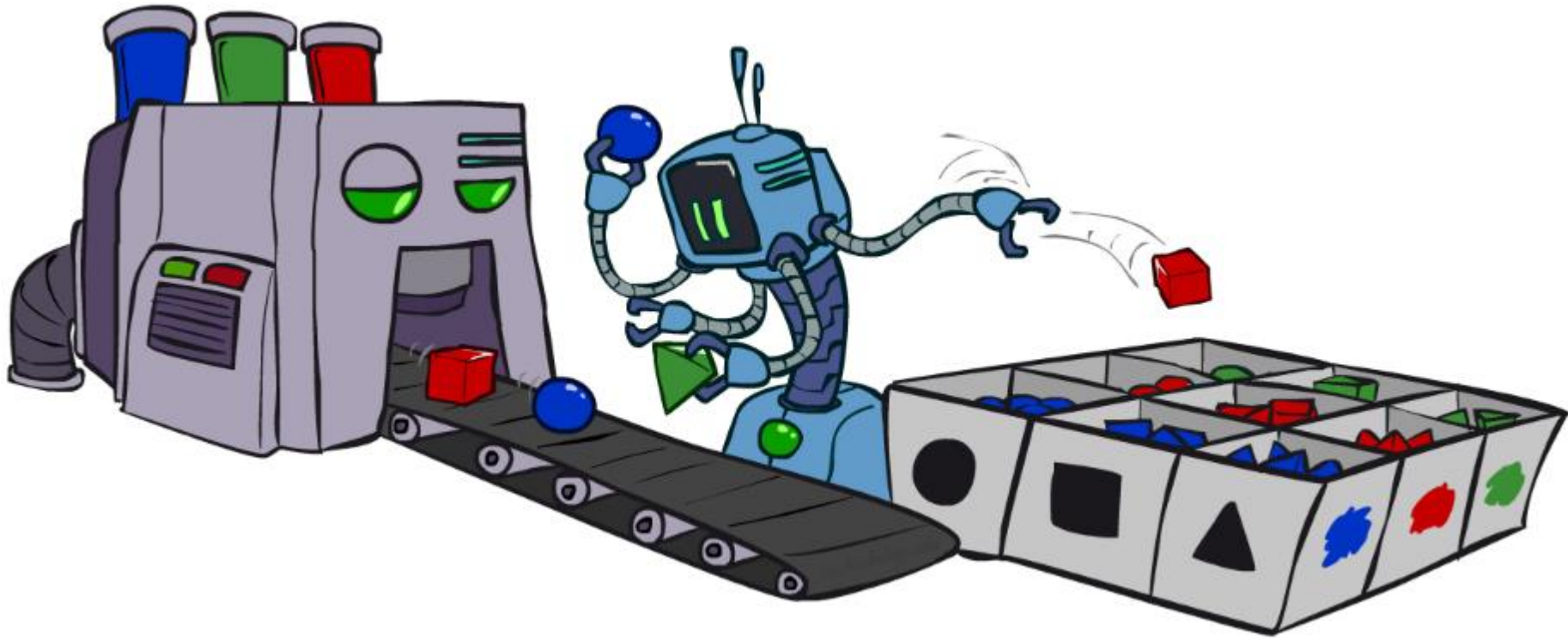
$$X_i \not\perp\!\!\!\perp X_j \mid \{X_{k_1}, \dots, X_{k_n}\}$$

- Otherwise (i.e. if all paths are inactive), then independence is guaranteed

$$X_i \perp\!\!\!\perp X_j \mid \{X_{k_1}, \dots, X_{k_n}\}$$



Bayes' Nets: Sampling



Sampling

- Sampling from given distribution

- Step 1: Get sample u from uniform distribution over $[0, 1)$

```
>>> import random
>>> random.random()
0.6303136415860905
```

- Step 2: Convert this sample u into an outcome for the given distribution by having each outcome associated with a sub-interval of $[0,1)$ with sub-interval size equal to probability of the outcome

- Example

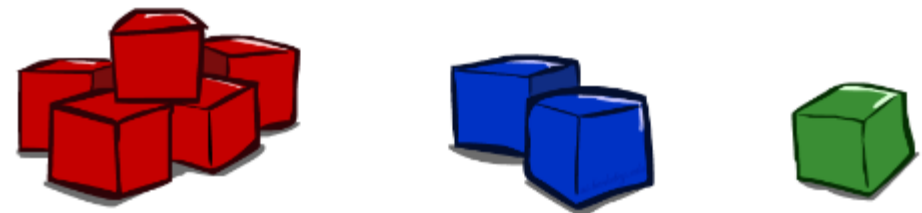
C	P(C)
red	0.6
green	0.1
blue	0.3

$0 \leq u < 0.6, \rightarrow C = \text{red}$

$0.6 \leq u < 0.7, \rightarrow C = \text{green}$

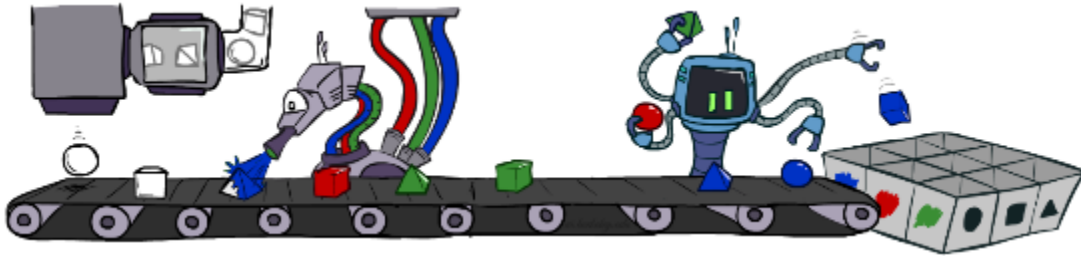
$0.7 \leq u < 1, \rightarrow C = \text{blue}$

- If `random()` returns $u = 0.83$, then our sample is $C = \text{blue}$
- E.g, after sampling 8 times:



Bayes' Net Sampling Summary

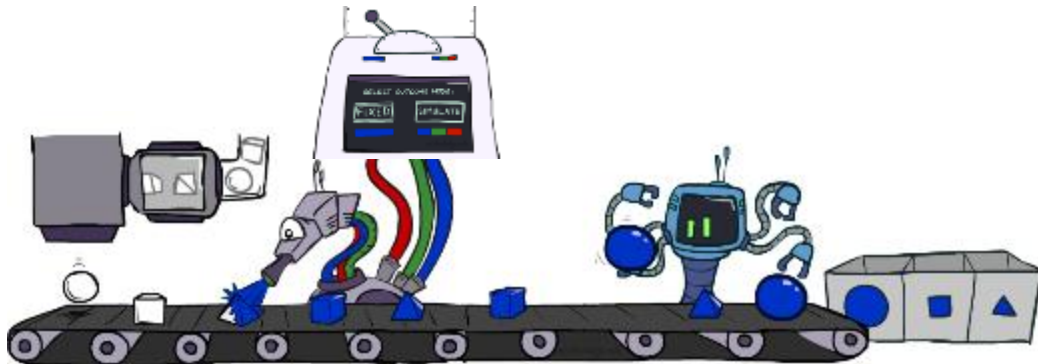
- Prior Sampling P



- Rejection Sampling $P(Q | e)$



- Likelihood Weighting $P(Q | e)$

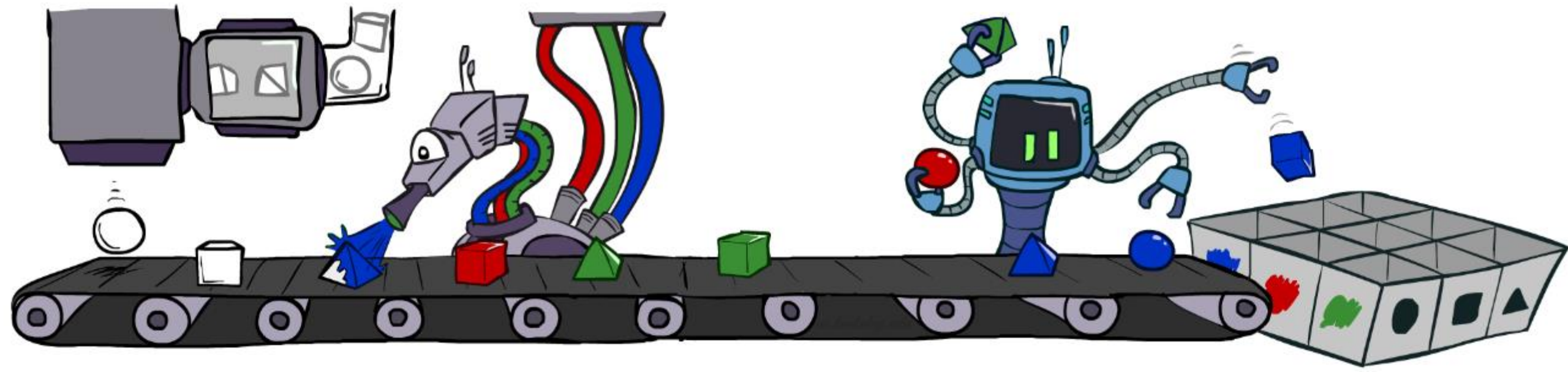


- Gibbs Sampling $P(Q | e)$



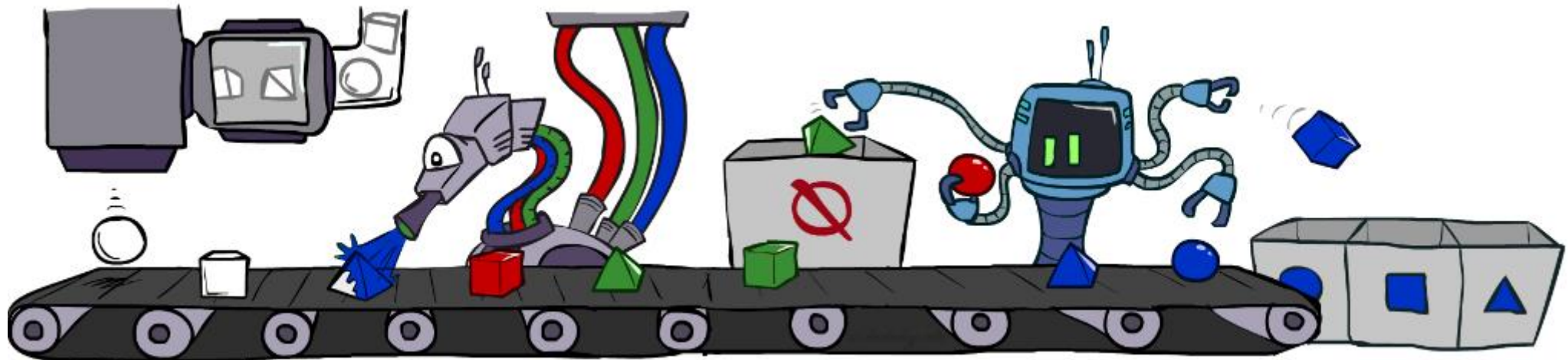
Prior Sampling

- For $i=1, 2, \dots, n$
 - Sample x_i from $P(X_i \mid \text{Parents}(X_i))$
- Return $(x_1, x_2, \dots, x_n)$



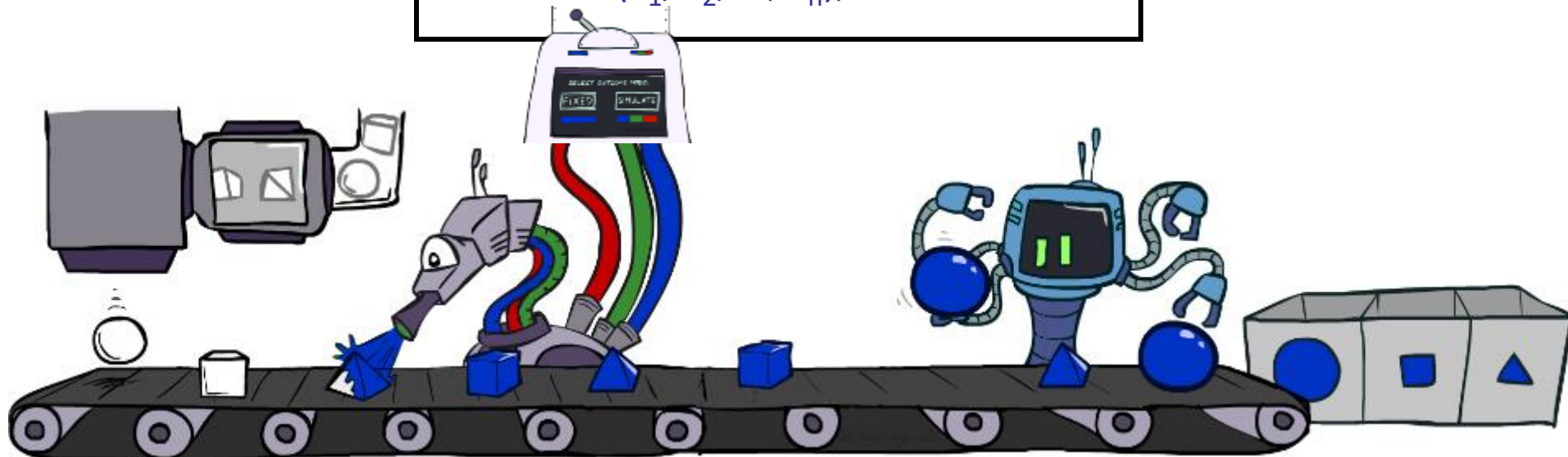
Rejection Sampling

- IN: evidence instantiation
- For $i=1, 2, \dots, n$
 - Sample x_i from $P(X_i \mid \text{Parents}(X_i))$
 - If x_i not consistent with evidence
 - Reject: Return, and no sample is generated in this cycle
- Return $(x_1, x_2, \dots, x_n)$



Likelihood Weighting

- IN: evidence instantiation
- $w = 1.0$
- for $i=1, 2, \dots, n$
 - if X_i is an evidence variable
 - $X_i = \text{observation } x_i \text{ for } X_i$
 - Set $w = w * P(x_i \mid \text{Parents}(X_i))$
 - else
 - Sample x_i from $P(X_i \mid \text{Parents}(X_i))$
- return $(x_1, x_2, \dots, x_n), w$

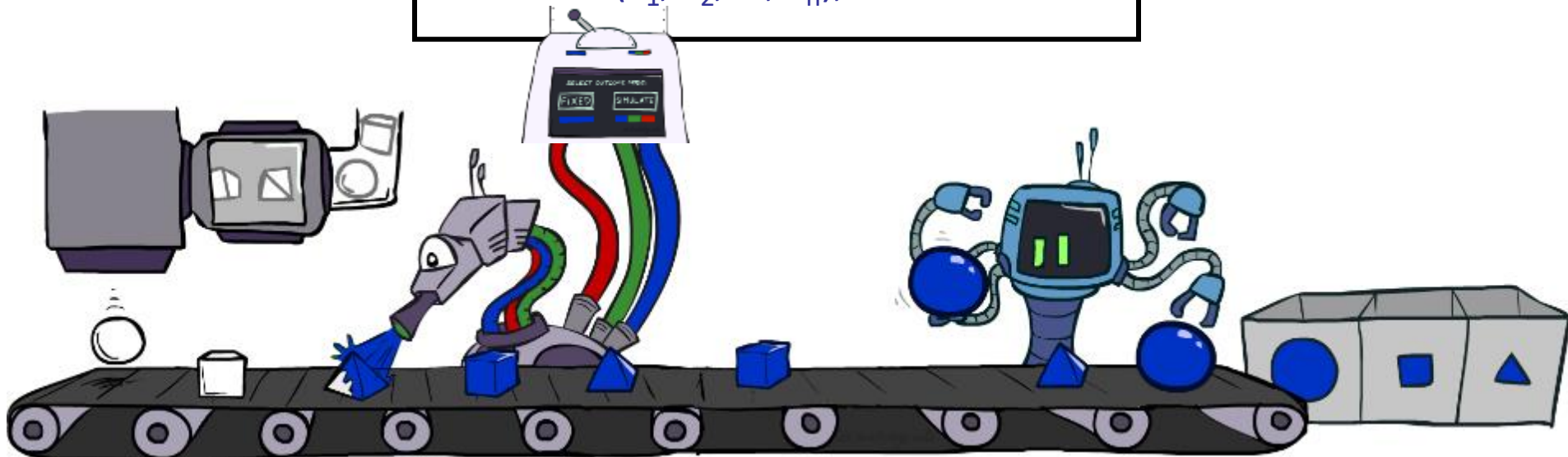


Likelihood Weighting

- IN: evidence instantiation
- $w = 1.0$
- for $i=1, 2, \dots, n$
 - if X_i is an evidence variable
 - $X_i = \text{observation } x_i \text{ for } X_i$
 - Set $w = w * P(x_i \mid \text{Parents}(X_i))$
 - else
 - Sample x_i from $P(X_i \mid \text{Parents}(X_i))$
- return $(x_1, x_2, \dots, x_n), w$

Now each sample doesn't count as 1.0 but has a weight. Need to take a weighted average.

$P(Q|\text{Evidence}) = \frac{\text{Sum}(\text{weights of samples consistent with Query})}{\text{Total Weight of All samples.}}$



Markov Models Recap

- Explicit assumption for all t : $X_t \perp\!\!\!\perp X_1, \dots, X_{t-2} \mid X_{t-1}$
- Consequence, joint distribution can be written as:

$$P(X_1, X_2, \dots, X_T) = P(X_1)P(X_2|X_1)P(X_3|X_2) \dots P(X_T|X_{T-1})$$

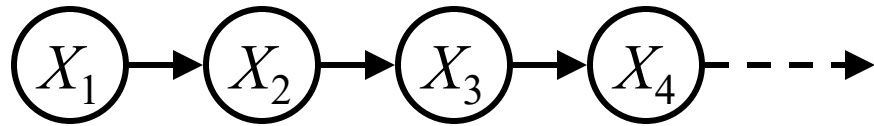
$$= P(X_1) \prod_{t=2}^T P(X_t|X_{t-1})$$

Huge savings in number of parameters needed!

- Implied conditional independencies:
 - Past variables independent of future variables given the present
i.e., if $t_1 < t_2 < t_3$ or $t_1 > t_2 > t_3$ then: $X_{t_1} \perp\!\!\!\perp X_{t_3} \mid X_{t_2}$
- Additional explicit assumption: $P(X_t \mid X_{t-1})$ is the same for all t

Mini-Forward Algorithm

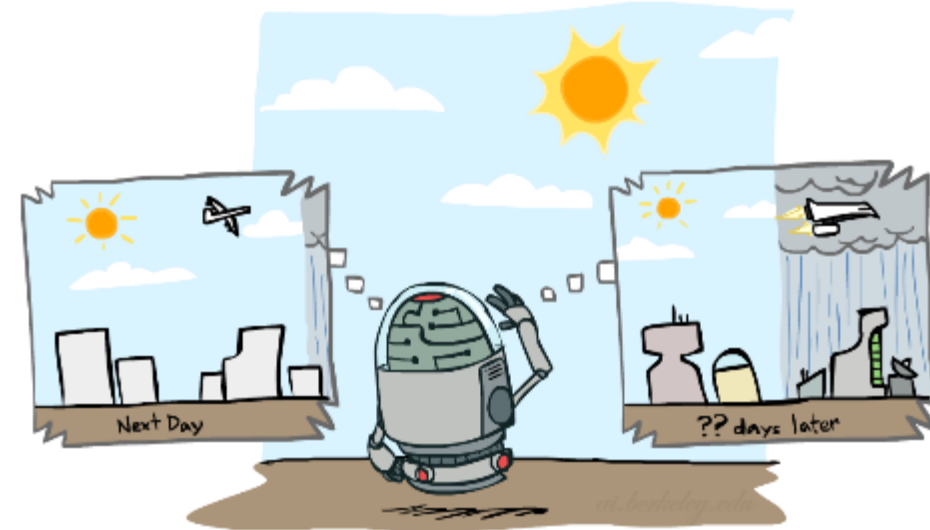
- Question: What's $P(X)$ on some day t ?



$$P(x_1) = \text{known}$$

$$\begin{aligned} P(x_t) &= \sum_{x_{t-1}} P(x_{t-1}, x_t) \\ &= \sum_{x_{t-1}} P(x_t \mid x_{t-1}) P(x_{t-1}) \end{aligned}$$

Forward simulation



Stationary Distributions

- For most chains:

- Influence of the initial distribution gets less and less over time.
- The distribution we end up in is independent of the initial distribution

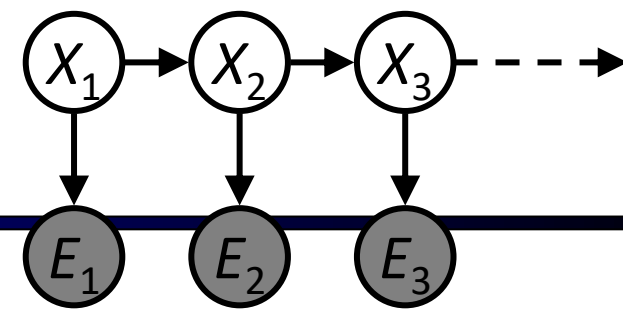
- Stationary distribution:

- The distribution we end up with is called the **stationary distribution** P_∞ of the chain
- It satisfies

$$P_\infty(X) = P_{\infty+1}(X) = \sum_x P(X|x)P_\infty(x)$$



HMMs Recap



- Explicit assumption for all t : $X_t \perp\!\!\!\perp X_1, \dots, X_{t-2} \mid X_{t-1}$
- Consequence, joint distribution can be written as:

$$\begin{aligned} P(X_1, X_2, \dots, X_T) &= P(X_1)P(X_2|X_1)P(X_3|X_2) \dots P(X_T|X_{T-1}) \\ &= P(X_1) \prod_{t=2}^T P(X_t|X_{t-1}) \end{aligned}$$

- Implied conditional independencies:
 - Past variables independent of future variables given the present
i.e., if $t_1 < t_2 < t_3$ or $t_1 > t_2 > t_3$ then: $X_{t_1} \perp\!\!\!\perp X_{t_3} \mid X_{t_2}$
- Additional explicit assumption: $P(X_t \mid X_{t-1})$ is the same for all t

The Forward Algorithm

- We are given evidence at each time and want to know

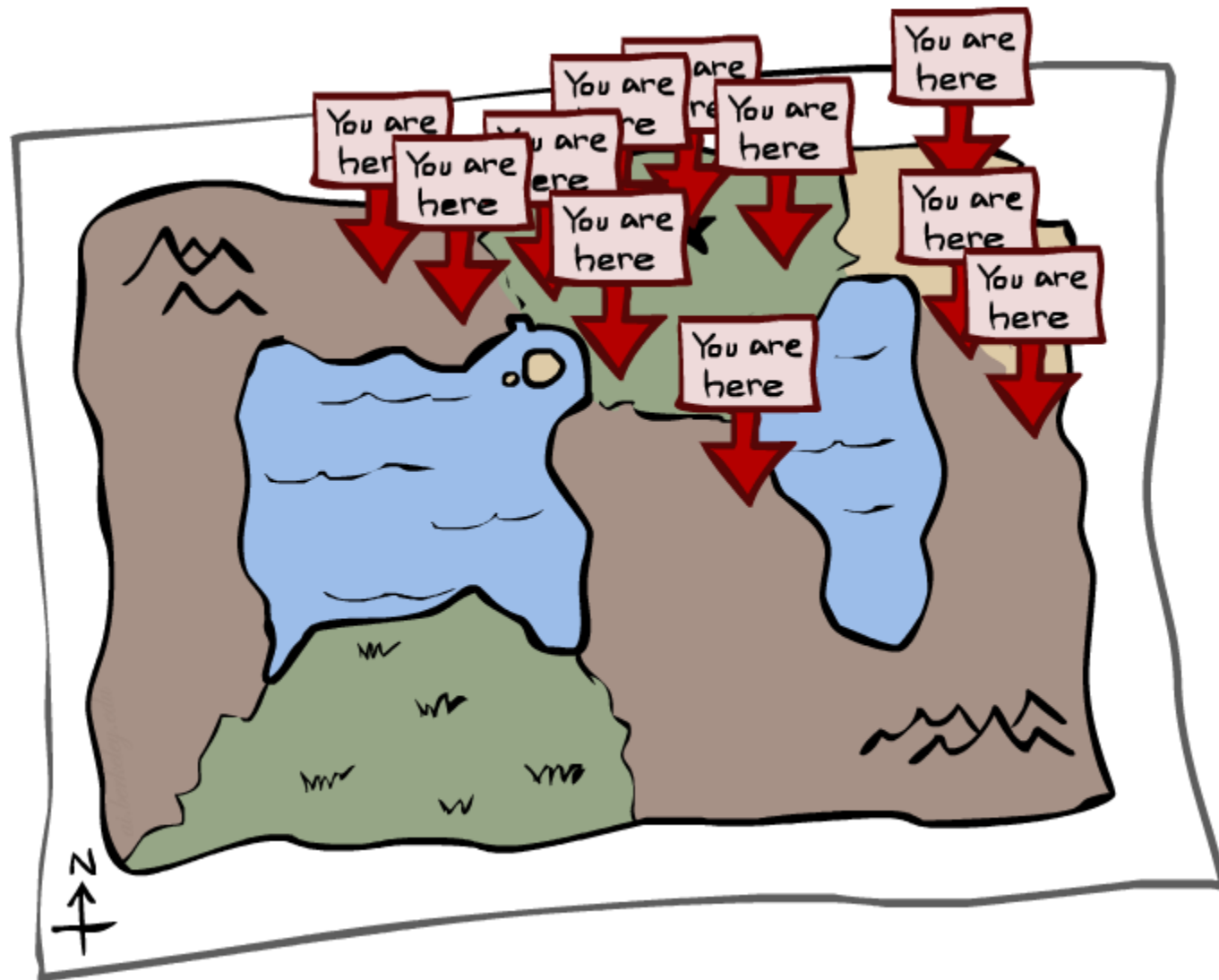
$$B_t(X) = P(X_t|e_{1:t})$$

- We can derive the following recursive update

$$P(X_t|e_{1:t}) = P(e_t|X_t) \sum_{x_{t-1}} P(X_t|x_{t-1}) P(X_{t-1}|e_{1:t-1})$$

$$B_t(X) = P(e_t|X_t) \sum_{x_{t-1}} P(X_t|x_{t-1}) B_{t-1}(X)$$

Particle Filtering



Particle Filtering

- Filtering: approximate solution
- Sometimes $|X|$ is too big to use exact inference
 - $|X|$ may be too big to even store $B(X)$
 - E.g. X is continuous
- Solution: approximate inference
 - Track samples of X , not all values
 - Samples are called particles
 - Time per step is linear in the number of samples
 - But: number needed may be large
 - In memory: list of particles, not states
- This is how robot localization works in practice
- Particle is just new name for sample

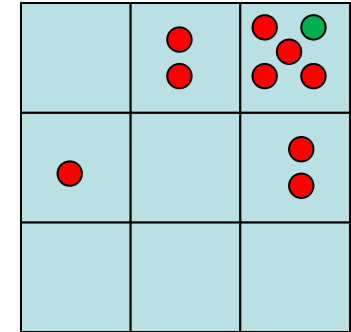
0.0	0.1	0.0
0.0	0.0	0.2
0.0	0.2	0.5



	●	
		● ●
	● ●	● ● ● ●

Representation: Particles

- Our representation of $P(X)$ is now a list of N particles (samples)
 - Generally, $N \ll |X|$
 - Storing map from X to counts would defeat the point
- $P(x)$ approximated by number of particles with value x
 - So, many x may have $P(x) = 0$!
 - More particles, more accuracy
- For now, all particles have a weight of 1



Particles:

(3,3)
(2,3)
(3,3)
(3,2)
(3,3)
(3,2)
(1,2)
(3,3)
(3,3)
(2,3)

Particle Filtering: Elapse Time

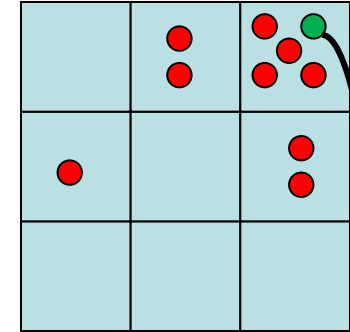
- Each particle is moved by sampling its next position from the transition model

$$x' = \text{sample}(P(X'|x))$$

- This is like prior sampling – samples' frequencies reflect the transition probabilities
- Here, most samples move clockwise, but some move in another direction or stay in place
- This captures the passage of time
 - If enough samples, close to exact values before and after (consistent)

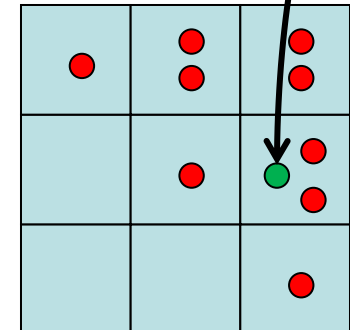
Particles:

(3,3)
(2,3)
(3,3)
(3,2)
(3,3)
(3,2)
(1,2)
(3,3)
(3,3)
(2,3)



Particles:

(3,2)
(2,3)
(3,2)
(3,1)
(3,3)
(3,2)
(1,3)
(2,3)
(3,2)
(2,2)



Particle Filtering: Observe

- Slightly trickier:

- Don't sample observation, fix it
- Similar to likelihood weighting, downweight samples based on the evidence

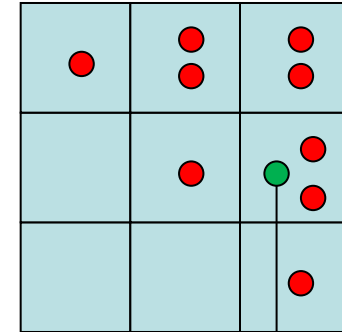
$$w(x) = P(e|x)$$

$$B(X) \propto P(e|X)B'(X)$$

- As before, the probabilities don't sum to one, since all have been downweighted (in fact they now sum to (N times) an approximation of $P(e)$)

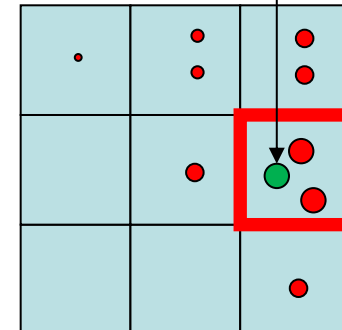
Particles:

(3,2)
(2,3)
(3,2)
(3,1)
(3,3)
(3,2)
(1,3)
(2,3)
(3,2)
(2,2)



Particles:

(3,2) w=.9
(2,3) w=.2
(3,2) w=.9
(3,1) w=.4
(3,3) w=.4
(3,2) w=.9
(1,3) w=.1
(2,3) w=.2
(3,2) w=.9
(2,2) w=.4



Particle Filtering: Resample

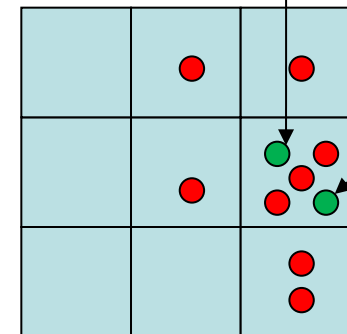
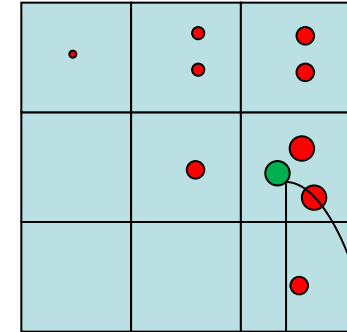
- Rather than tracking weighted samples, we resample
- N times, we choose from our weighted sample distribution (i.e. draw with replacement)
- This is equivalent to renormalizing the distribution
- Now the update is complete for this time step, continue with the next one

Particles:

(3,2) w=.9
(2,3) w=.2
(3,2) w=.9
(3,1) w=.4
(3,3) w=.4
(3,2) w=.9
(1,3) w=.1
(2,3) w=.2
(3,2) w=.9
(2,2) w=.4

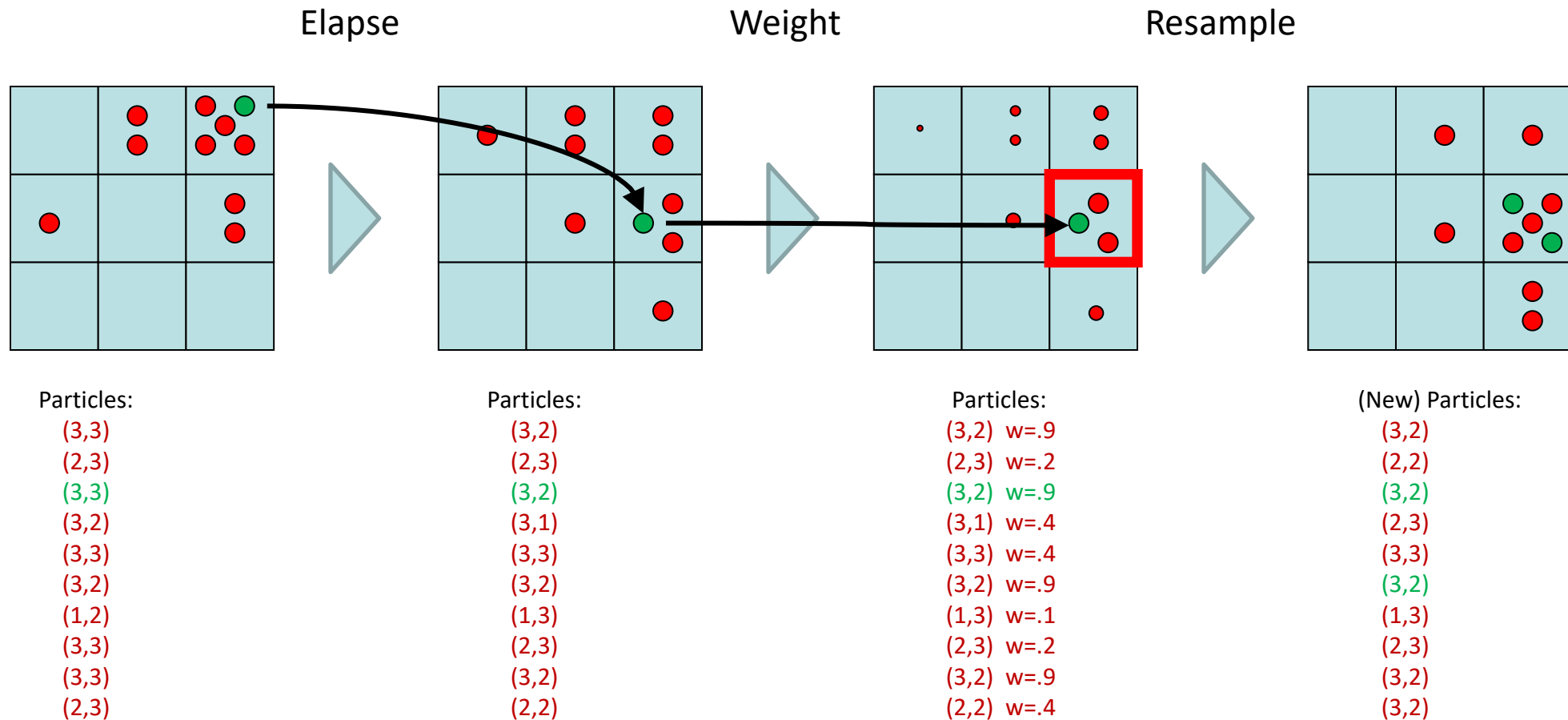
(New) Particles:

(3,2)
(2,2)
(3,2)
(2,3)
(3,3)
(3,2)
(1,3)
(2,3)
(3,2)
(3,2)

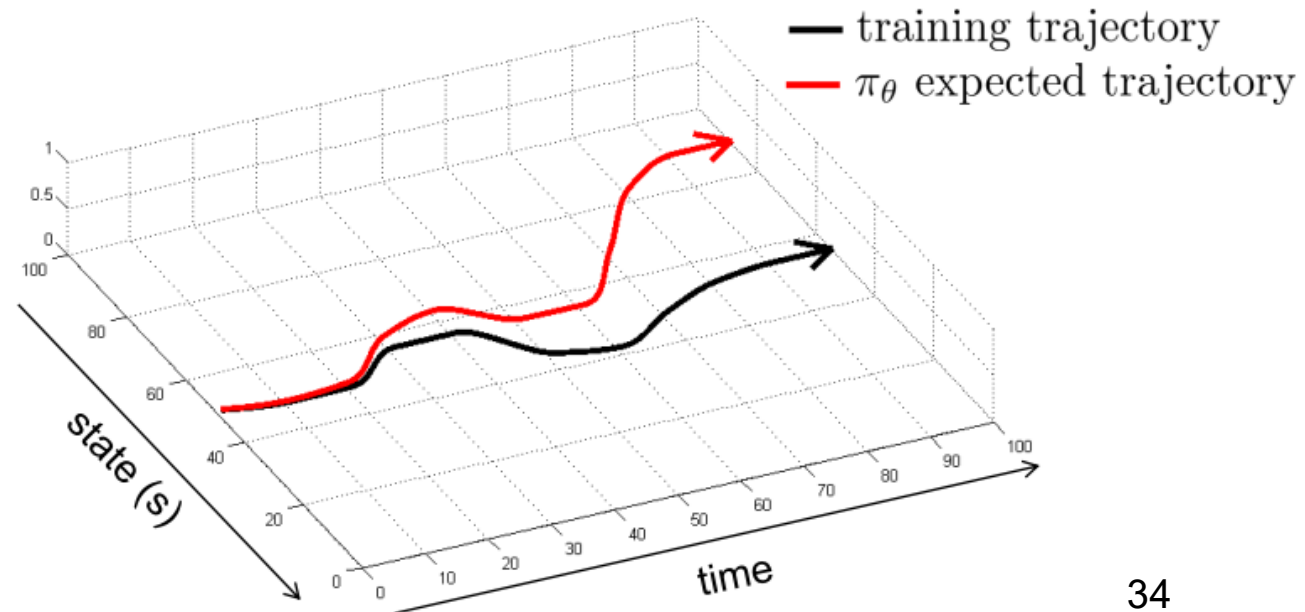
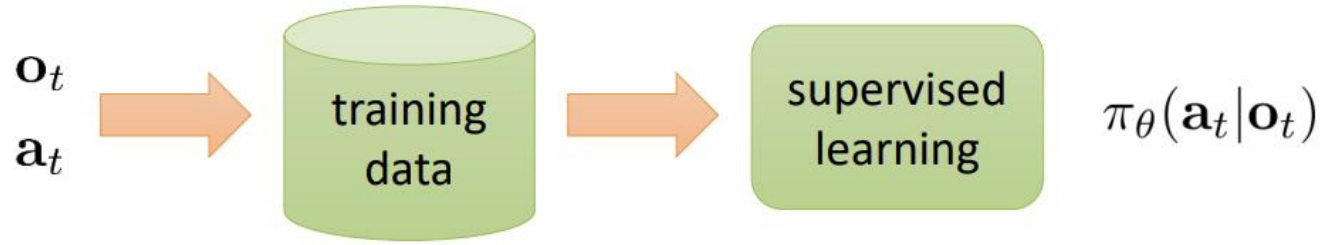


Recap: Particle Filtering

- Particles: track samples of states rather than an explicit distribution

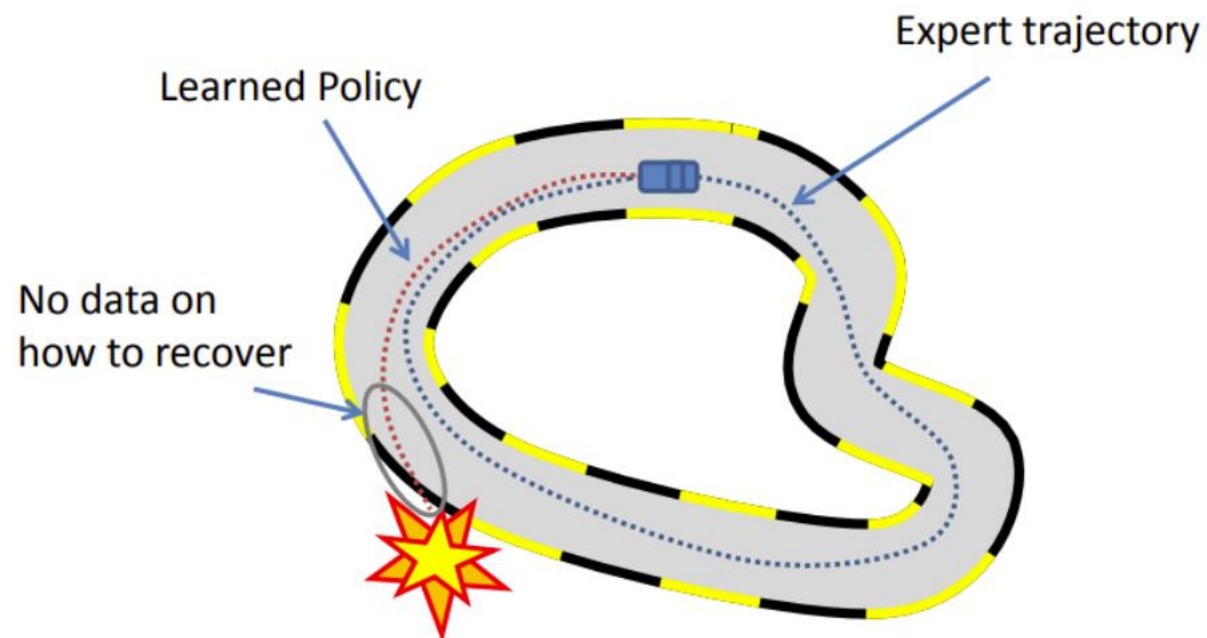


Behavioral Cloning



Distribution Shift

$$p_{\pi^*}(o_t) \neq p_{\pi_\theta}(o_t)$$



	Supervised Learning	Supervised Learning + Control
Train	$(x, y) \sim D$	$s \sim P(\cdot s, \pi^*(s))$
Test	$(x, y) \sim D$	$s \sim P(\cdot s, \pi(s))$

DAgger

can we make $p_{\text{data}}(\mathbf{o}_t) = p_{\pi_\theta}(\mathbf{o}_t)$?

idea: instead of being clever about $p_{\pi_\theta}(\mathbf{o}_t)$, be clever about $p_{\text{data}}(\mathbf{o}_t)$!

DAgger: Dataset Aggregation

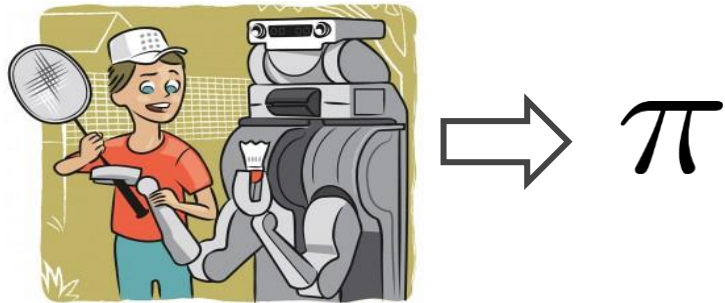
goal: collect training data from $p_{\pi_\theta}(\mathbf{o}_t)$ instead of $p_{\text{data}}(\mathbf{o}_t)$

how? just run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$

but need labels $\mathbf{a}_t$!

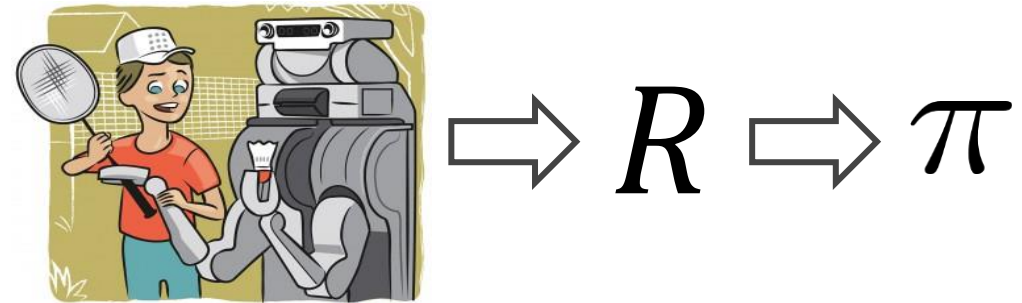
- 
1. train $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ from human data $\mathcal{D} = \{\mathbf{o}_1, \mathbf{a}_1, \dots, \mathbf{o}_N, \mathbf{a}_N\}$
 2. run $\pi_\theta(\mathbf{a}_t|\mathbf{o}_t)$ to get dataset $\mathcal{D}_\pi = \{\mathbf{o}_1, \dots, \mathbf{o}_M\}$
 3. Ask human to label $\mathcal{D}_\pi$ with actions $\mathbf{a}_t$
 4. Aggregate: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_\pi$

Behavioral Cloning



- Answers the “How?” question
- Mimic the demonstrator
- Learn mapping from states to actions
- Computationally efficient
- Compounding errors

Inverse Reinforcement Learning

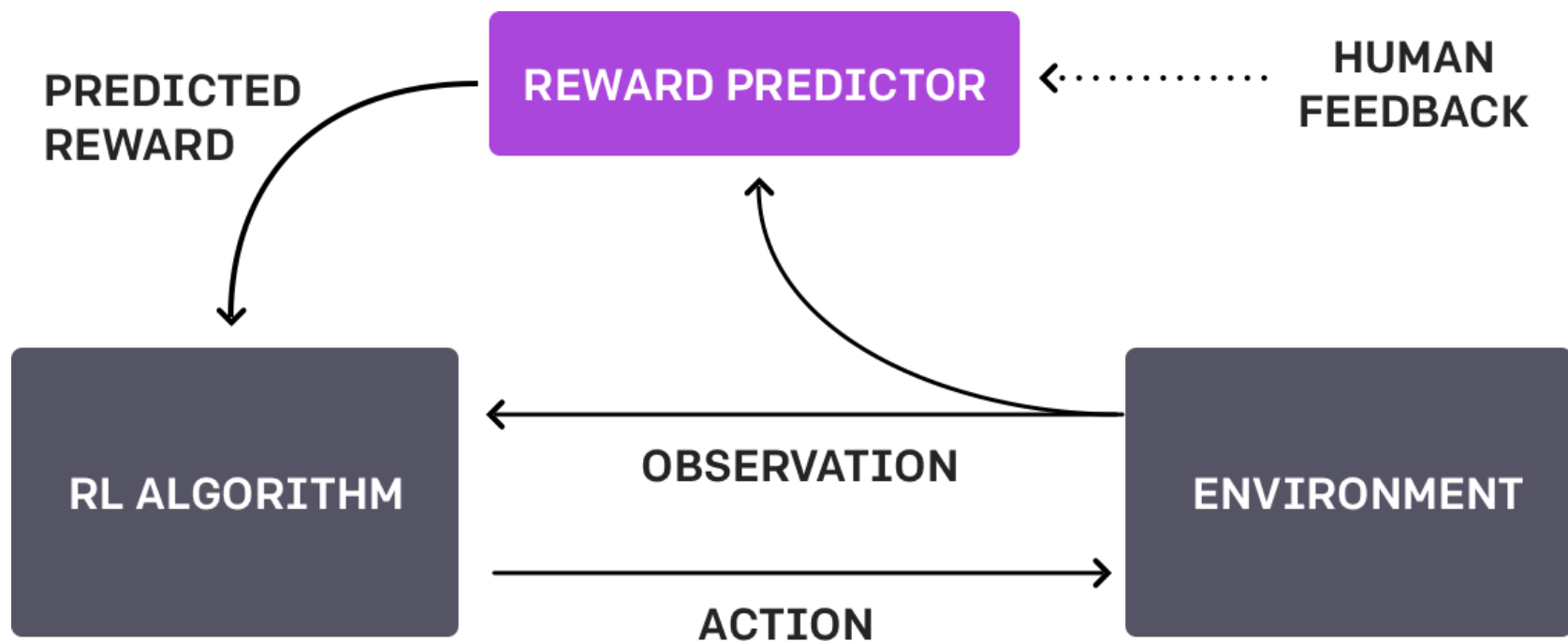


- Answers the “Why?” question
- Explain the demonstrator’s behavior
- Learn a reward function capturing the demonstrator’s intent
- Can require lots of data and compute
- Better generalization. Can recover from arbitrary states

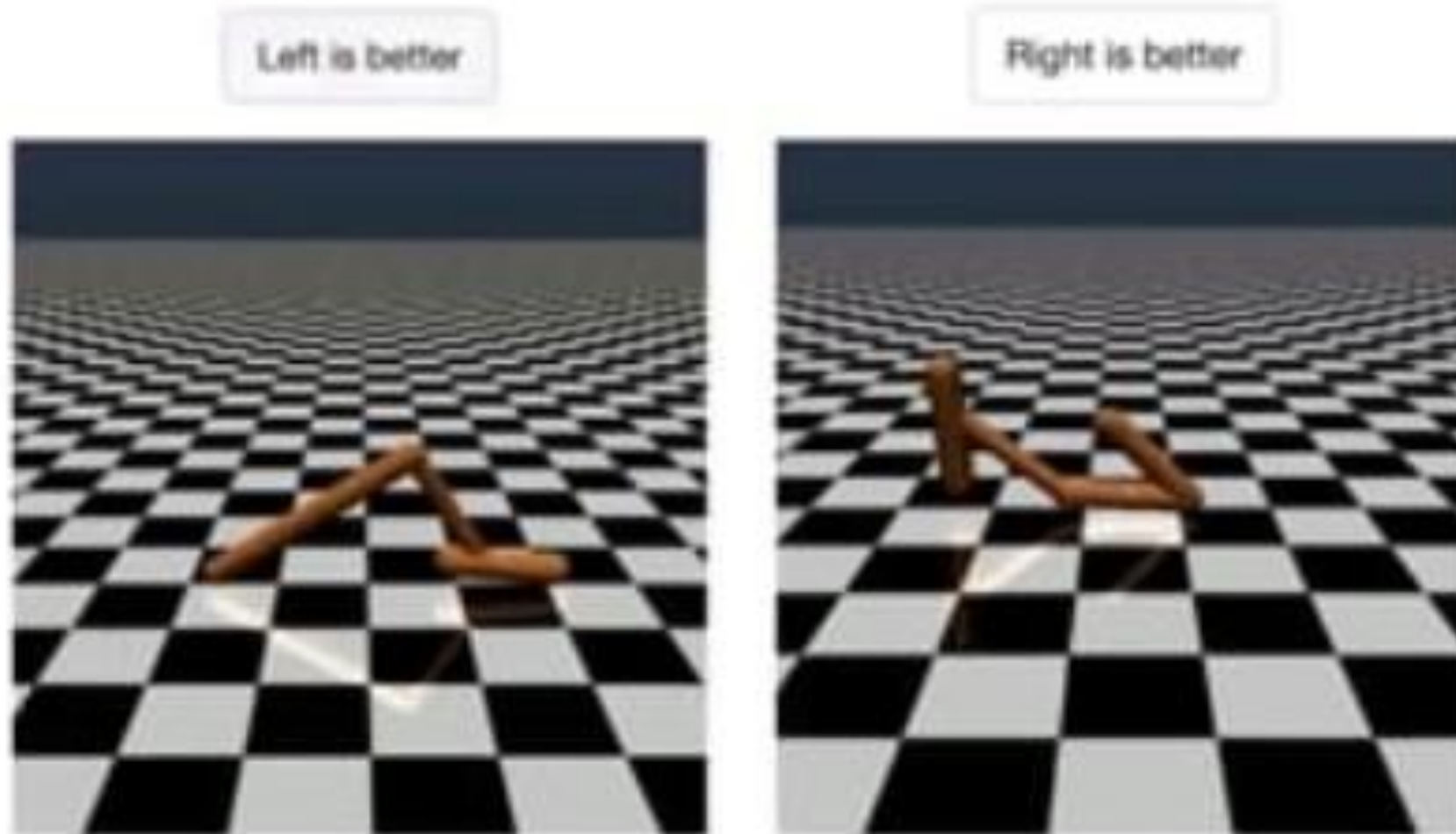
Basic IRL Algorithm

- ⌘ Start with demonstrations, D
- ⌘ Guess initial reward function R_0
- ⌘ $\hat{R} = R_0$
- ⌘ Loop:
 - ✧ Solve for optimal policy $\pi_{\hat{R}}^*$
 - ✧ Compare D and $\pi_{\hat{R}}^*$
 - ✧ Update $\hat{R}$ to try and make D and $\pi_{\hat{R}}^*$ more similar

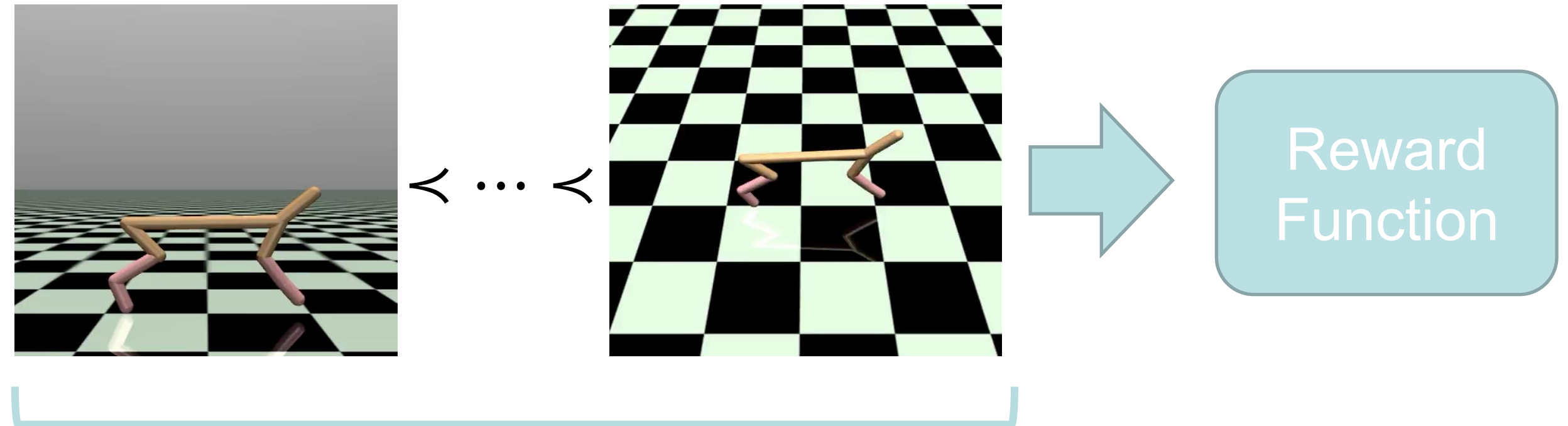
RL from Human Feedback (RLHF)



RL from Human Preferences

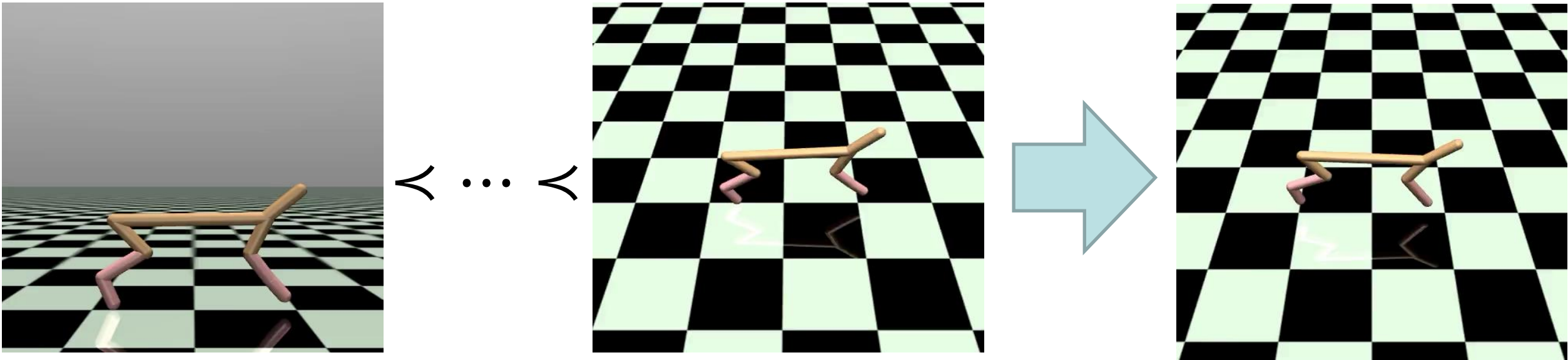


RLHF



Pre-ranked demonstrations

RLHF



Pre-ranked demonstrations

T-REX Policy

Learning from preferences

$$\tau_1 \prec \tau_2 \prec \cdots \prec \tau_T$$

$$\sum_{s \in \tau_1} R_\theta(s) < \sum_{s \in \tau_2} R_\theta(s)$$

Bradley-Terry pairwise ranking
loss

$$\mathcal{L}(\theta) = - \sum_{\tau_i \prec \tau_j} \frac{\exp \sum_{s \in \tau_j} R_\theta(s)}{\exp \sum_{s \in \tau_i} R_\theta(s) + \exp \sum_{s \in \tau_j} R_\theta(s)}$$

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



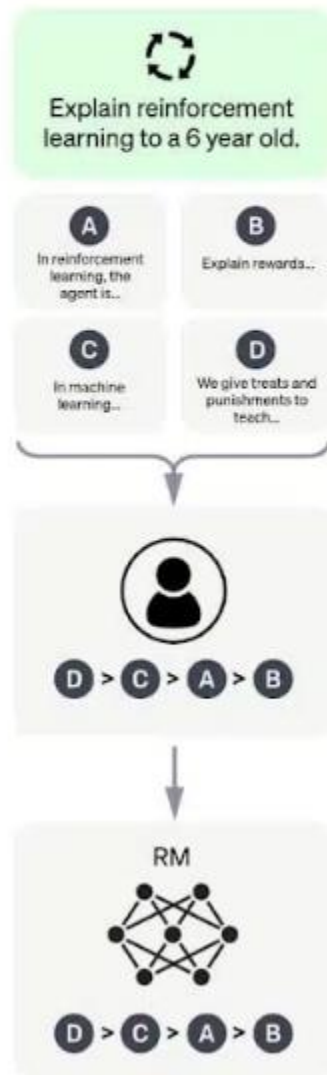
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

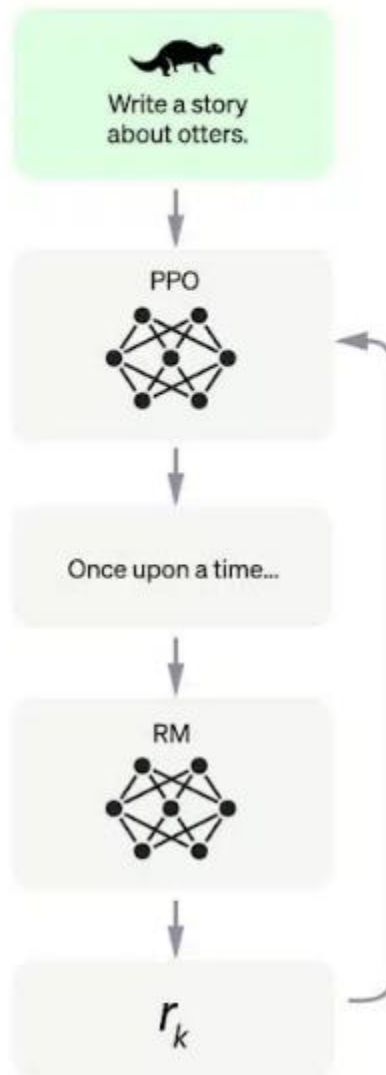
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



We made it!
